Solving the Theory of Equality and Uninterpreted Functions

Theory and Practice of SMT Solving

Bruno Andreotti

21 September 2026

Universidade Federal de Minas Gerais

Review: the EUF theory

- In a previous lecture, we talked about the theory of *equality and uninterpreted functions* (EUF)

- Formally, the axioms of this theory are built into the equational first-order logic we use in SMT
  - which is why it is sometimes called the "empty" theory

Example:

$$f(f(a)) \neq b \wedge a = b \wedge a = f(a)$$

Review: the EUF theory

Axioms of equality:

$$\forall a \,.\, a = a \qquad \text{(reflexivity)}$$

$$\forall a, b \,.\, a = b \leftrightarrow b = a \qquad \text{(symmetry)}$$

$$\forall a, b, c \,.\, (a = b \wedge b = c) \rightarrow a = c \qquad \text{(transitivity)}$$

Additional axiom:

$$\forall a, b \,.\, a = b \rightarrow f(a) = f(b) \qquad \text{(congruence)}$$

Formal definition of the EUF problem

- When solving an SMT problem that involves the EUF theory, the solver will consider all equality terms as atoms, and search for a model

- From the perspective of the theory solver, this model will be a series of equalities or disequalities, and we must determine if they are consistent with the EUF axioms or not

Formal definition of the EUF problem

Example:

$$\neg(f(f(a)) = b) \wedge a = b \wedge a = f(a)$$

- Satisfying model: $f(f(a)) = b \mapsto \bot, \quad a = b \mapsto \top, \quad a = f(a) \mapsto \top$

Formal definition of the EUF problem

Example:

$$\neg(f(f(a)) = b) \wedge a = b \wedge a = f(a)$$

- Satisfying model: $f(f(a)) = b \mapsto \bot, \quad a = b \mapsto \top, \quad a = f(a) \mapsto \top$

- EUF problem: the problem of determining whether a set of equalities $\mathcal{E}$ and a set of disequalities $\mathcal{D}$ are consistent with the EUF axioms.

$$\mathcal{E} = \{a = b,\ a = f(a)\} \quad \mathcal{D} = \{f(f(a)) \neq b\}$$

Congruence closure

Definition: congruence closure

Given a relation $\mathcal{R}$, there exists a minimal extension $\mathcal{R}^*$ such that $\mathcal{R}^*$ is an equivalence relation, and is *closed under congruences*.

We say that $\mathcal{R}^*$ is the **congruence closure** of $\mathcal{R}$.

Congruence closure

Definition: congruence closure

Given a relation $\mathcal{R}$, there exists a minimal extension $\mathcal{R}^*$ such that $\mathcal{R}^*$ is an equivalence relation, and is *closed under congruences*.

We say that $\mathcal{R}^*$ is the **congruence closure** of $\mathcal{R}$.

- An EUF problem instance $\{\mathcal{E}, \mathcal{D}\}$ is consistent if and only if the congruence closure of $\mathcal{E}$ does not contain any equalities between terms in $\mathcal{D}$.

$$\forall a, b \,.\, (a \neq b) \in \mathcal{D} \rightarrow (a = b) \notin \mathcal{E}^*$$

Congruence closure

- This means we can:
  1. compute the set of **equivalence classes** of the congruence clousre of $\mathcal{E}$,

Congruence closure

- This means we can:
  1. compute the set of **equivalence classes** of the congruence clousre of $\mathcal{E}$,
  2. and check that all the disequalities in $\mathcal{D}$ respect the equivalence classes.

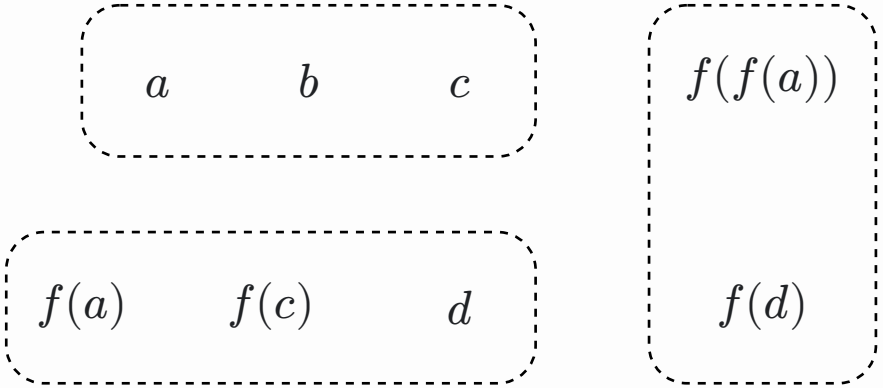
Congruence closure

$$
\begin{array}{c}
a = b \\
b = c \\
d = f(c) \\
f(d) \neq f(f(a))
\end{array}
\quad \Longrightarrow \quad
\begin{array}{cccc}
a & b & c & f(f(a)) \\
f(a) & f(c) & d & f(d)
\end{array}
$$

Congruence closure

$$a = b$$
$$b = c$$
$$d = f(c)$$
$$f(d) \neq f(f(a))$$

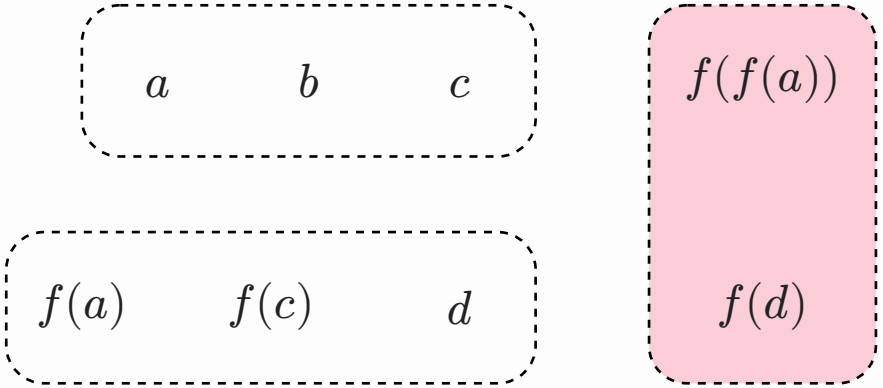
$\Longrightarrow$



Congruence closure

$$a = b$$
$$b = c$$
$$d = f(c)$$
$$f(d) \neq f(f(a))$$

$\Longrightarrow$



Congruence closure

$$
\begin{aligned}
a &= b \\
b &= c \\
d &= f(c) \\
f(d) &\neq f(f(a))
\end{aligned}
\quad \Longrightarrow \quad
\begin{array}{ccc}
a & b & c \\
f(a) & f(c) & d
\end{array}
\qquad
\begin{array}{c}
f(f(a)) \\
\\
f(d)
\end{array}
$$

- This will be the general shape of the algorithm I will describe today, but first we need to do some pre-processing!

Currying

- It's complicated to deal with functions that can have any number of arguments
- Instead, we transfrom these terms into their **Curry form**

Currying

- We are going to introduce a new “apply” function symbol, denoted “$\cdot$”, that takes two arguments

- We represent every function term by applying $\cdot$ to the function and the argument, recursively

Currying

Example:

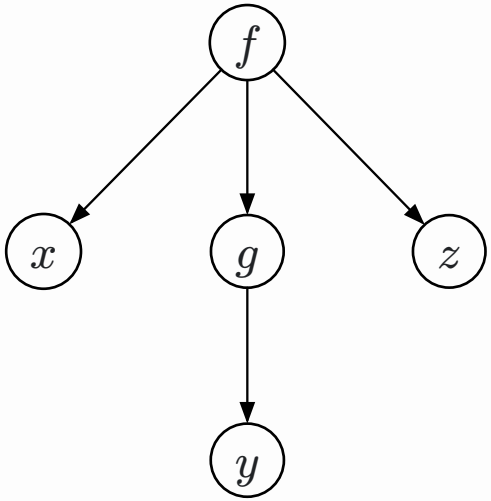
$$f(x) \mapsto \bullet(f, x)$$
$$f(x, y, z) \mapsto \bullet(\bullet(\bullet(f, x), y), z)$$
$$x \mapsto x$$

Definition:

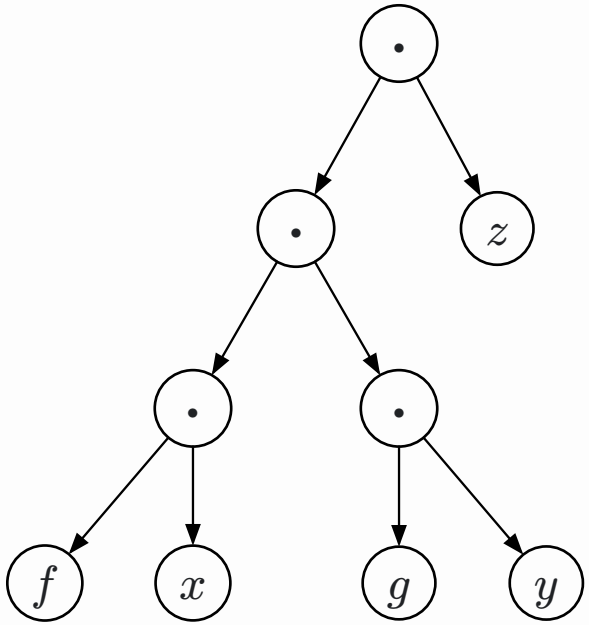
$$\begin{aligned} \text{curry}(x) &= x \\ \text{curry}(f(x_1, ..., x_n)) &= \bullet(\text{curry}(f(x_1, ..., x_{n-1})), x_n) \\ &= \bullet(...\bullet(\bullet(f, x_1), x_2), ... \ x_n) \end{aligned}$$

Currying

$f(x, g(y), z)$



$\bullet(\bullet(\bullet(f, x), \bullet(g, y)), z)$



Computing congruence closure

- At the start, each term will be in its own equivalence class
- We will compute the congruence closure by iterating over the input equalities, and merging the equivalence classes as needed
- After processing all equalities, we will have constructed the congruence closure induced by them

Computing congruence closure

- *pending*: The list of input equalities we have not yet processed
- *representative*(t) or t': For each term t, stores a term r which is a unique representative of the equivalence class of t.
  - At the start, $t' = t$
- *class*(r): For each representative, stores a set with all the terms in its class
- *lookup*(a, b): For each terms a, b, *lookup*(a, b) returns a term equivalent to $\bullet(a, b)$, or *null* if none exists
- *useList*(r): For each representative r, store a set of all terms $\bullet(x, y)$ where $x' = r$ or $y' = r$

Computing congruence closure

```
1  function congruenceClosure(ℰ):
2      pending ← ℰ
3      while pending ≠ ∅:
4          take a = b from pending
5          if a′ ≠ b′:
6              merge(a, b)
```

```
1  function merge(a, b):
       // assume |class(a′)| < |class(b′)|
2      for each c ∈ class(a′):
3          c′ ← b′
4          class(b′) ← class(b′) ∪ {c}
5      for each e = •(c, d) ∈ useList(a′):
6          let f = lookup(c′, d′)
7          if f ≠ null ∧ f′ ≠ e′:
8              pending ← pending ∪ {e′ = f′}
9          lookup(c′, d′) ← e′
10         useList(b′) ← useList(b′) ∪ {•(c, d)}
```

Example

$$\mathcal{E} = \left\{ \begin{array}{c} f(a) = g(b) \\ g(c) = h(d, g(a)) \\ b = c \\ h(d, d) = g(b) \end{array} \right\}$$

Example

$$\mathcal{E} = \left\{ \begin{array}{rl} \bullet(f,a) &= \bullet(g,b) \\ \bullet(g,c) &= \bullet(\bullet(h,d),\bullet(g,a)) \\ b &= c \\ \bullet(\bullet(h,d),d) &= \bullet(g,b) \end{array} \right\}$$

Example

$$\text{pending} = \left\{ \begin{array}{r} t_8 = t_{10} \\ t_{11} = t_{14} \\ t_2 = t_3 \\ t_{13} = t_{10} \end{array} \right\}$$

	term	repr.	useList
1	a		$\{8, 9\}$
2	b		$\{10\}$
3	c		$\{11\}$
4	d		$\{12, 13\}$
5	f		$\{8\}$
6	g		$\{9, 10, 11\}$
7	h		$\{12\}$
8	$\bullet(f, a)$		$\{\}$
9	$\bullet(g, a)$		$\{14\}$
10	$\bullet(g, b)$		$\{\}$
11	$\bullet(g, c)$		$\{\}$
12	$\bullet(h, d)$		$\{13, 14\}$
13	$\bullet(\bullet(h, d), d)$		$\{\}$
14	$\bullet(\bullet(h, d), \bullet(g, a))$		$\{\}$

Example

$$\text{pending} = \left\{ \begin{matrix} t_{11} = t_{14} \\ t_2 = t_3 \\ t_{13} = t_{10} \end{matrix} \right\}$$

	term	repr.	useList
1	a		$\{8, 9\}$
2	b		$\{10\}$
3	c		$\{11\}$
4	d		$\{12, 13\}$
5	f		$\{8\}$
6	g		$\{9, 10, 11\}$
7	h		$\{12\}$
8	$\bullet(f, a)$	10	$\{\}$
9	$\bullet(g, a)$		$\{14\}$
10	$\bullet(g, b)$		$\{\}$
11	$\bullet(g, c)$		$\{\}$
12	$\bullet(h, d)$		$\{13, 14\}$
13	$\bullet(\bullet(h, d), d)$		$\{\}$
14	$\bullet(\bullet(h, d), \bullet(g, a))$		$\{\}$

Example

$$\text{pending} = \left\{ \begin{matrix} t_2 = t_3 \\ t_{13} = t_{10} \end{matrix} \right\}$$

	term	repr.	useList
1	a		$\{8, 9\}$
2	b		$\{10\}$
3	c		$\{11\}$
4	d		$\{12, 13\}$
5	f		$\{8\}$
6	g		$\{9, 10, 11\}$
7	h		$\{12\}$
8	$\bullet(f, a)$	10	$\{\}$
9	$\bullet(g, a)$		$\{14\}$
10	$\bullet(g, b)$		$\{\}$
11	$\bullet(g, c)$	14	$\{\}$
12	$\bullet(h, d)$		$\{13, 14\}$
13	$\bullet(\bullet(h, d), d)$		$\{\}$
14	$\bullet(\bullet(h, d), \bullet(g, a))$		$\{\}$

Example

$$\text{pending} = \{t_{13} = t_{10}\}$$

	term	repr.	useList
1	a		$\{8, 9\}$
2	b	3	$\{10\}$
3	c		$\{11\}$
4	d		$\{12, 13\}$
5	f		$\{8\}$
6	g		$\{9, 10, 11\}$
7	h		$\{12\}$
8	$\bullet(f, a)$	10	$\{\}$
9	$\bullet(g, a)$		$\{14\}$
10	$\bullet(g, b)$		$\{\}$
11	$\bullet(g, c)$	14	$\{\}$
12	$\bullet(h, d)$		$\{13, 14\}$
13	$\bullet(\bullet(h, d), d)$		$\{\}$
14	$\bullet(\bullet(h, d), \bullet(g, a))$		$\{\}$

Example

$$\text{pending} = \begin{Bmatrix} t_{13} = t_{10} \\ t_{10} = t_{14} \end{Bmatrix}$$

	term	repr.	useList
1	a		$\{8, 9\}$
2	b	3	$\{10\}$
3	c		$\{11, 10\}$
4	d		$\{12, 13\}$
5	f		$\{8\}$
6	g		$\{9, 10, 11\}$
7	h		$\{12\}$
8	$\bullet(f, a)$	10	$\{\}$
9	$\bullet(g, a)$		$\{14\}$
10	$\bullet(g, b)$		$\{\}$
11	$\bullet(g, c)$	14	$\{\}$
12	$\bullet(h, d)$		$\{13, 14\}$
13	$\bullet(\bullet(h, d), d)$		$\{\}$
14	$\bullet(\bullet(h, d), \bullet(g, a))$		$\{\}$

Example

$$\text{pending} = \{t_{10} = t_{14}\}$$

	term	repr.	useList
1	a		$\{8, 9\}$
2	b	3	$\{10\}$
3	c		$\{11, 10\}$
4	d		$\{12, 13\}$
5	f		$\{8\}$
6	g		$\{9, 10, 11\}$
7	h		$\{12\}$
8	$\bullet(f, a)$	10	$\{\}$
9	$\bullet(g, a)$		$\{14\}$
10	$\bullet(g, b)$		$\{\}$
11	$\bullet(g, c)$	14	$\{\}$
12	$\bullet(h, d)$		$\{13, 14\}$
13	$\bullet(\bullet(h, d), d)$	14	$\{\}$
14	$\bullet(\bullet(h, d), \bullet(g, a))$		$\{\}$

Example

$\text{pending} = \{\}$

	term	repr.	useList
1	a		$\{8, 9\}$
2	b	3	$\{10\}$
3	c		$\{11, 10\}$
4	d		$\{12, 13\}$
5	f		$\{8\}$
6	g		$\{9, 10, 11\}$
7	h		$\{12\}$
8	$\bullet(f, a)$	14	$\{\}$
9	$\bullet(g, a)$		$\{14\}$
10	$\bullet(g, b)$	14	$\{\}$
11	$\bullet(g, c)$	14	$\{\}$
12	$\bullet(h, d)$		$\{13, 14\}$
13	$\bullet(\bullet(h, d), d)$	14	$\{\}$
14	$\bullet(\bullet(h, d), \bullet(g, a))$		$\{\}$

Example

- At the end:

$$\left\{\begin{array}{c} \bullet(f,a) \\ \bullet(g,a) \\ \bullet(g,b) \\ \bullet(g,c) \\ \bullet(\bullet(h,d),d) \\ \bullet(\bullet(h,d),\bullet(g,a)) \end{array}\right\} \quad \left\{\begin{array}{c} b \\ c \end{array}\right\}$$

...and all other terms are in singleton classes

Complexity analysis

```
1  function congruenceClosure(E):
2      pending ← E
3      while pending ≠ ∅:
4          take a = b from pending
5          if a′ ≠ b′:
6              merge(a, b)
```

```
1  function merge(a, b):
       // assume |class(a′)| < |class(b′)|
2      for each c ∈ class(a′):
3          c′ ← b′
4          class(b′) ← class(b′) ∪ {c}
5      for each e = •(c, d) ∈ useList(a′):
6          let f = lookup(c′, d′)
7          if f ≠ null ∧ f′ ≠ e′:
8              pending ← pending ∪ {e′ = f′}
9          lookup(c′, d′) ← e′
10         useList(b′) ← useList(b′) ∪ {•(c, d)}
```

Complexity analysis

- The *lookup* function can be implemented using a hash table or array, so accessing it is $O(1)$
- Each term can only change representative up to $O(\log n)$ times[1], so the total time spent updating the *representative* table is $O(\log n)$
- Similarly, each input equation $e = \bullet\,(c, d)$ can only change *useList* $O(\log n)$ times.
- In total, the complexity to construct the congruence closure is $O(n \log n)$

[1] note that the class size always at least doubles after a merge

Complexity analysis

- We must also consider the complexity of checking if two terms are congruent
- To do this, we just compute the representatives of each term, and compare them. Since the depth of the tree is bounded $O(\log n)$, this takes $O(\log n)$ time.
- In total, performing the queries to see if the at most n input disequalities are consistent takes $O(n \log n)$ time.

Complexity analysis

- As such, the total complexity of this algorithm is $O(n \log n)$.
- Notably, this is very cheap compared to most other theory solvers

Complexity analysis

- As such, the total complexity of this algorithm is $O(n \log n)$.

- Notably, this is very cheap compared to most other theory solvers
  - linear integer arithmetic: $O(2^n)$
  - non-linear real arithmetic: $O(2^{2^n})$
  - strings: **undecidable!**

Some historical context

Fast Decision Procedures Based on Congruence Closure

GREG NELSON AND DEREK C. OPPEN

Stanford University, Stanford, California

Congruence Closure with Integer Offsets

Robert Nieuwenhuis* and Albert Oliveras**

Technical University of Catalonia
Jordi Girona 1
08034 Barcelona, Spain
{roberto,oliveras}@lsi.upc.es

Variations on the Common Subexpression Problem

PETER J. DOWNEY

The University of Arizona, Tucson, Arizona

RAVI SETHI

Bell Laboratories, Murray Hill, New Jersey

AND

ROBERT ENDRE TARJAN

Stanford University, Stanford, California

Some historical context

Fast Decision Procedures Based on Congruence Closure

GREG NELSON AND DEREK C. OPPEN

Stanford University, Stanford, California

Variations on the Common Subexpression Problem

PETER J. DOWNEY

The University of Arizona, Tucson, Arizona

RAVI SETHI

Bell Laboratories, Murray Hill, New Jersey

AND

ROBERT ENDRE TARJAN

Stanford University, Stanford, California

Congruence Closure with Integer Offsets

Robert Nieuwenhuis* and Albert Oliveras**

Technical University of Catalonia
Jordi Girona 1
08034 Barcelona, Spain
{roberto,oliveras}@lsi.upc.es

Proof-producing Congruence Closure

Robert Nieuwenhuis* and Albert Oliveras**

Technical University of Catalonia, Jordi Girona 1, 08034 Barcelona, Spain
www.lsi.upc.es/~roberto www.lsi.upc.es/~oliveras

Proofs and explanations

- So far, we have described an algorithm that can answer **“yes”** or **“no”** to whether two terms are equivalent under congruence

- However, in some cases this boolean answer might not be good enough

Proofs and explanations

- For example, consider the case where the input literals are:

$$\mathcal{E} = \{a = f(b),\ \ b = c,\ x_1 = y_1,\ x_2 = y_2,\ \ldots,\ x_n = y_n\}$$
$$\mathcal{D} = \{f(c) \neq a\}$$

- If we simply tell the SAT solver that this model is inconsistent, the conflict clause will use **all** literals:

$$f(c) = a \ \vee\ a \neq f(b) \ \vee\ b \neq c \ \vee\ x_1 \neq y_1 \ \vee\ x_2 \neq y_2 \ \vee\ \cdots \ \vee\ x_n \neq y_n$$

Proofs and explanations

- Instead, it would be nice if we could convey to the solver that only the first three atoms are enough to form an inconsistent model

- To do this, we will modify our algorithm so that, when we determine that two terms are equivalent, we can also produce an **explanation** of their equivalence

Proofs and explanations

- Formally, an explanation is a minimal[1] set of equalities that is sufficient to make two terms equivalent

- In the example, the explanation for $f(c) \equiv a$ is $\{\ a = f(b), b = c\ \}$

- From this, we create the conflict clause

$$f(c) = a \ \vee \ a \neq f(b) \ \vee \ b \neq c$$

  which is much more useful

[1] as in, if you remove any equality from it, it doesn't work anymore

Proofs and explanations

- Besides their use as conflict clauses, explanations can also be used to construct **proofs** of unsatisfiability:

$$\dfrac{\dfrac{a = f(b) \qquad \dfrac{b = c}{f(b) = f(c)}\,\text{cong.}}{a = f(c)}\,\text{trans.}}{f(c) = a}\,\text{symm.}$$

Proofs and explanations

- Besides their use as conflict clauses, explanations can also be used to construct **proofs** of unsatisfiability:

$$\dfrac{\dfrac{a = f(b) \qquad \dfrac{b = c}{f(b) = f(c)}\,\text{cong.}}{a = f(c)}\,\text{trans.}}{f(c) = a}\,\text{symm.}$$

- These are crucial when you want to produce a proof of the unsatisfiability of the formula as a whole

Equality graph

- In addition to our congruence closure structure, we will construct an **equality graph**
- Each term will be a node in the graph, and the edges are the class merges that were done

Equality graph

Input: a $f(b)$

b c x $f(a)$

y

Equality graph

Input:

$a = b$

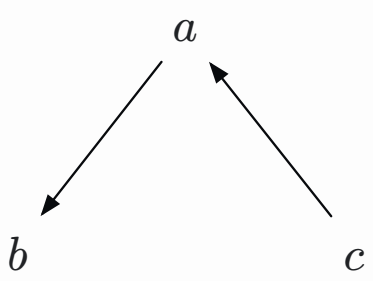


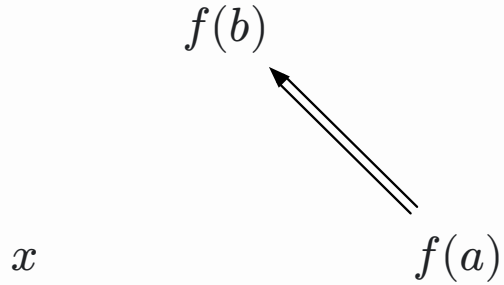
Equality graph

Input:

$a = b$

$c = a$





y

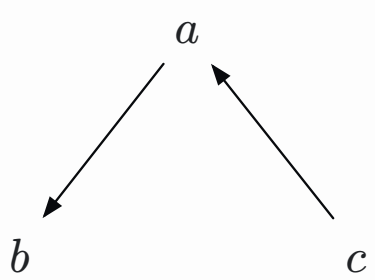
Equality graph

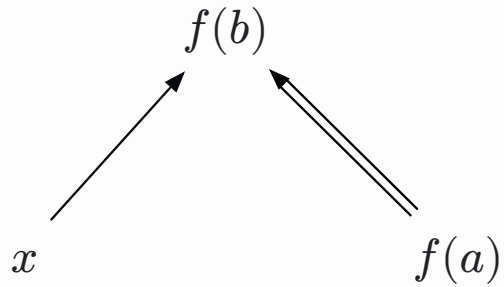
Input:

$a = b$

$c = a$

$x = f(b)$





y

Equality graph

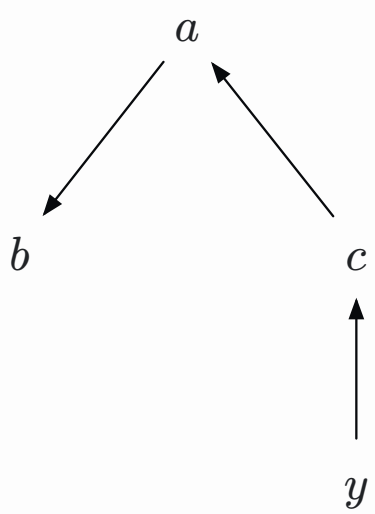
Input:

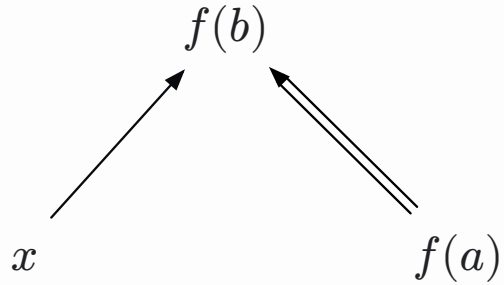
$a = b$

$c = a$

$x = f(b)$

$y = c$





Equality graph

Input:

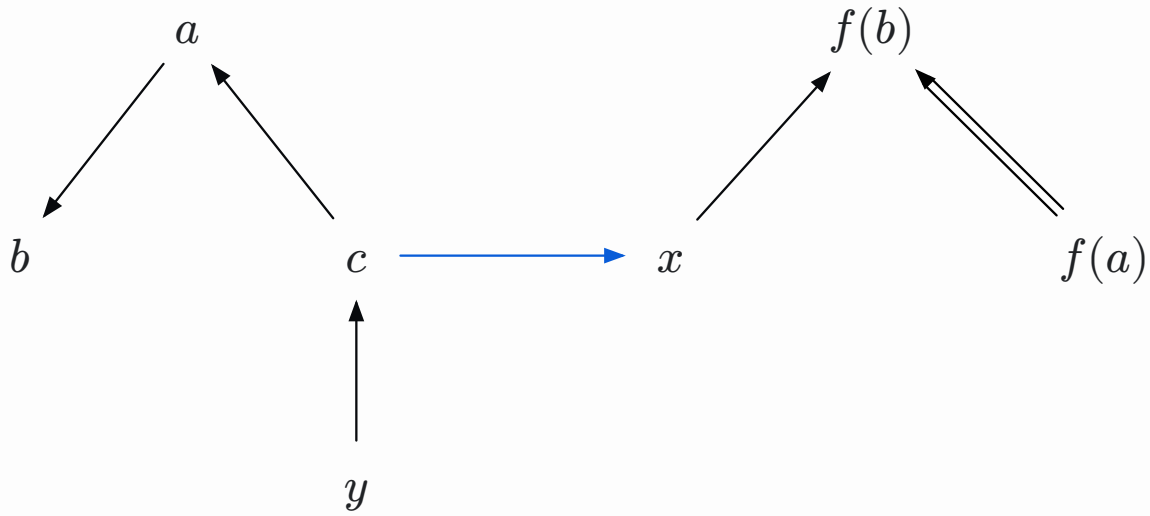
$a = b$

$c = a$

$x = f(b)$

$y = c$

$c = x$



Equality graph

Input:

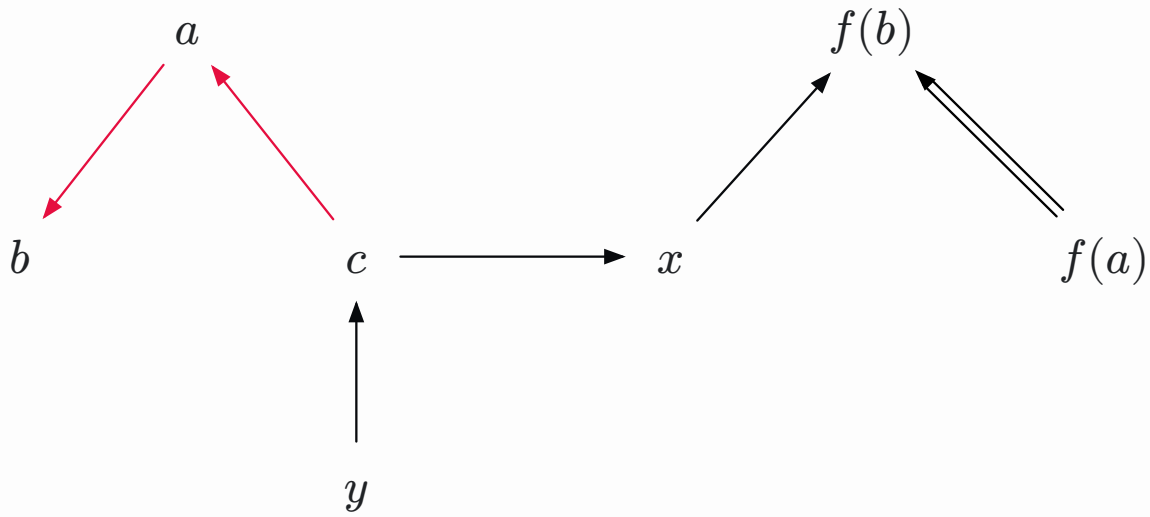
$a = b$

$c = a$

$x = f(b)$

$y = c$

$c = x$



Equality graph

Input:

$a = b$

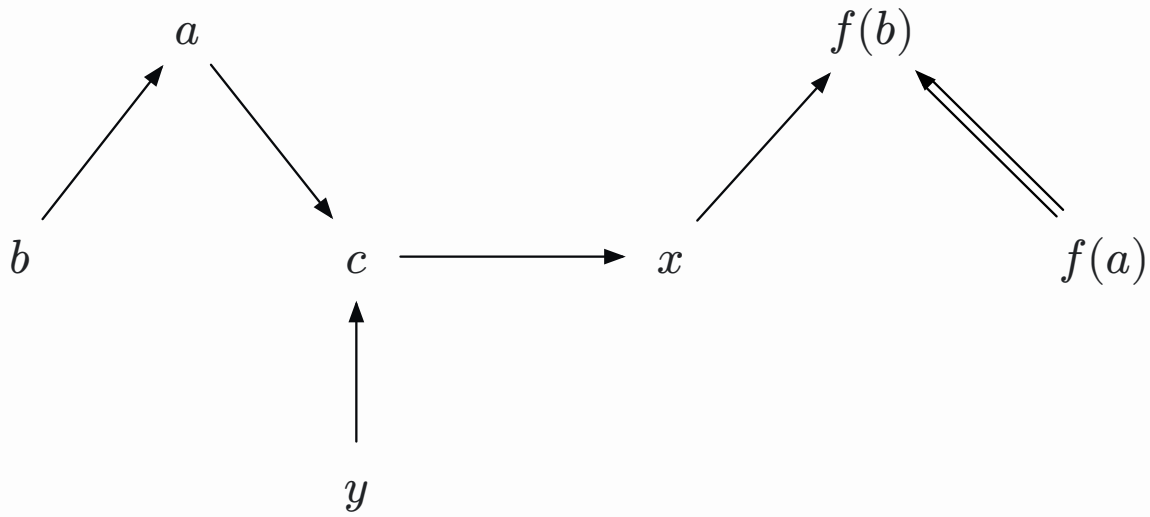
$c = a$

$x = f(b)$

$y = c$

$c = x$

$y \stackrel{?}{=} f(a)$



Equality graph

Input:

$a = b$

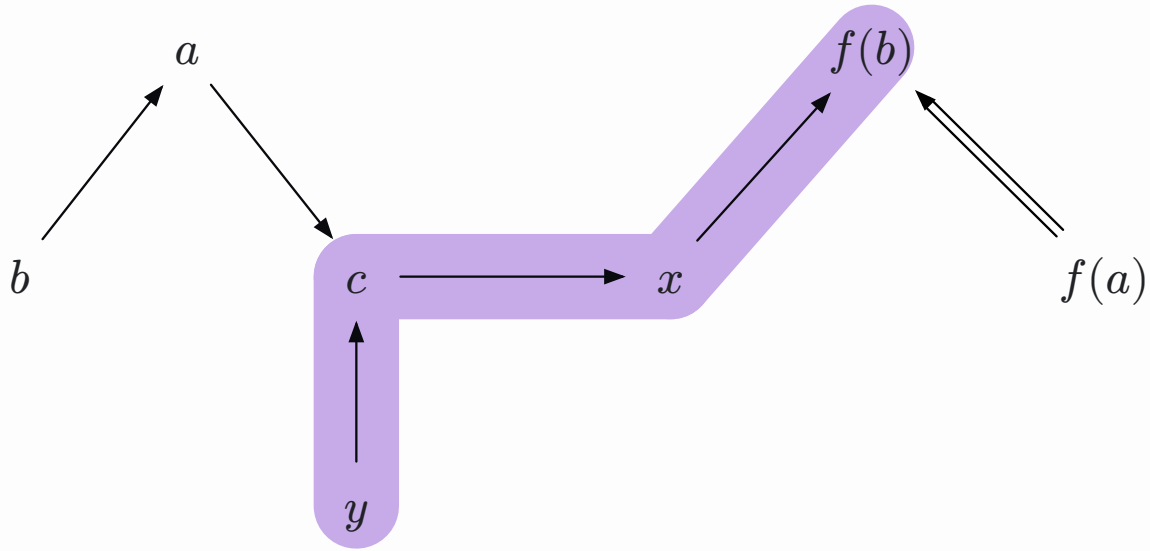
$c = a$

$x = f(b)$

$y = c$

$c = x$

$y \stackrel{?}{=} f(a)$



Equality graph

Input:

$a = b$

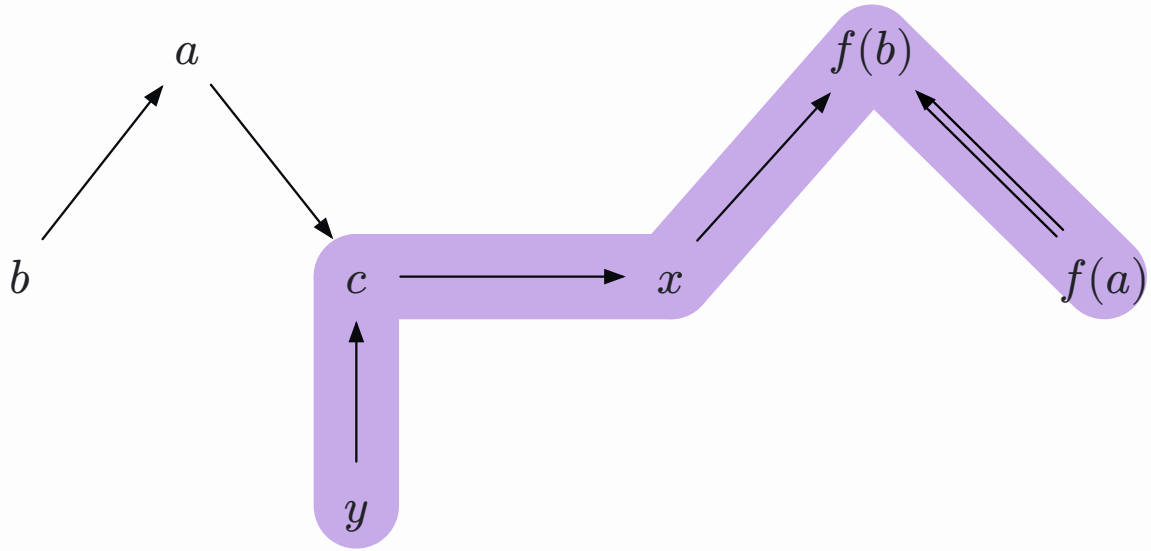
$c = a$

$x = f(b)$

$y = c$

$c = x$

$y \stackrel{?}{=} f(a)$



Equality graph

Input:

$a = b$

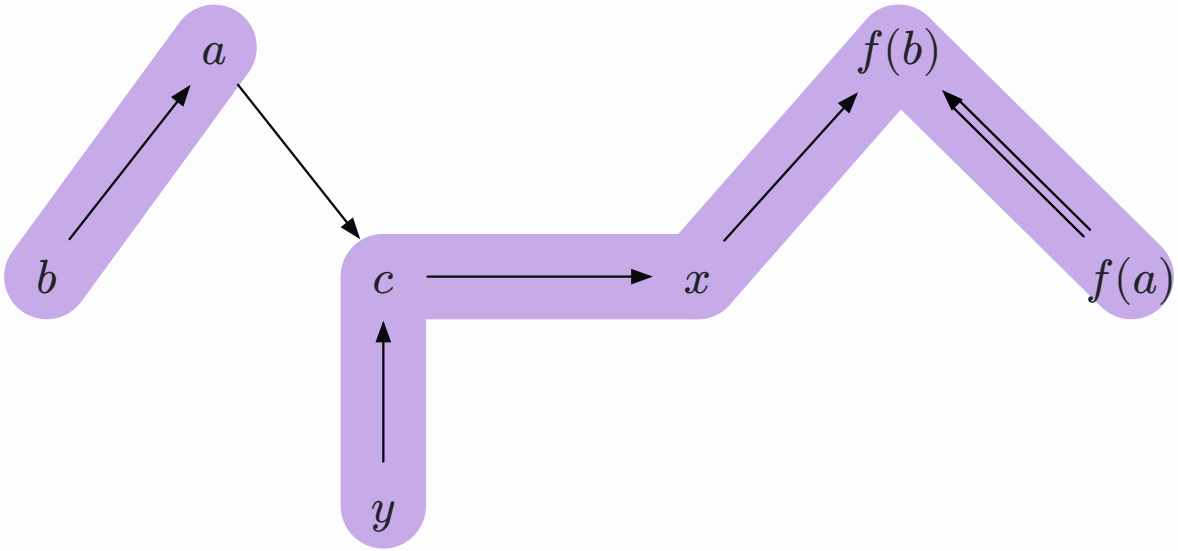
$c = a$

$x = f(b)$

$y = c$

$c = x$

$y \stackrel{?}{=} f(a)$



Equality graph

```
1 function explain(a, b):
2     let lca = lowestCommonAncestor(a, b)
3     return explainPath(a, lca) ∪
          explainPath(b, lca)
```

```
1  function explainPath(lower, upper):
2      let E = ∅
3      let a = lower
4      while a ≠ upper:
5          let b = parent(a)
6          if the edge a–b is f(a_1, a_2) = f(b_1, b_2):
               // the edge is a congruence edge
7              E ← E ∪ explain(a_1, b_1) ∪ explain(a_2, b_2)
8          else
               // the edge is a single input equality
9              E ← E ∪ {a = b}
10         a ← b
11     return E
```

Proof producing congruence closure

- Here, we've only shown the version of the algorithm that produces explanations

- However, it is not hard to instrument it to also produce structured proofs
  - each input equality you add to the explanation is an assumption to the proof
  - each congruence edge you visit becomes a congruence step
  - and we have to add transitivity steps to connect the path

Complexity analysis: *merge*

- With this modified algorithm, we must do some extra work when merging equivalence classes, as we may have to reorient edges up to the root of one of the merged classes.

Complexity analysis: *merge*

- With this modified algorithm, we must do some extra work when merging equivalence classes, as we may have to reorient edges up to the root of one of the merged classes.
- Since the proof graph is a forest, we have at most $n - 1$ edges, and each edge can be reoriented at most $O(\log n)$ times.
- So, the total time spent doing this extra work is $O(n \log n)$

Complexity analysis: *explain*

- The way we implemented *explain* is not the most efficient, as it may try to explain the same terms multiple times
- Solving this limitation is tricky, but can be done with the use of an additional union-find data structure
- With this optimization, the complexity of producing the explanation can be $O(k\,\alpha(k,k))$ (where k is the size of the final proof), which is bound by $O(n \log n)$

Conclusion

- Congruence closure is a fundamental part of modern SMT solvers
- The algorithms we saw today are used not only for the EUF theory, but as components in many other theory solvers (strings, arrays, etc.)

Conclusion

- Besides SMT solving, congruence closure/e-graphs are used in many other applications:
  - equality saturation
  - first-order theorem provers
  - compilers and program optimization
  - hardware design and synthesis
  - ...

Thanks!

Extra secret slides

- Remember how congruence closure is much cheaper ($O(n \log n)$) than other theory solvers?

Extra secret slides

- Remember how congruence closure is much cheaper ($O(n \log n)$) than other theory solvers?

- This allows the SMT solver to use the EUF engine much more often

Extra secret slides

- In particular, this theory is usually included in **theory propagation**, that is, the use of theory solvers during the propagation phase of CDCL(T)

- This allows the solver to reject invalid models early, while also deriving new literals from congruence closure